Assignment 2 Report

Parallel 3D Stencil and Isovalue Computation

Submitted by
Khusal Nikam (251110043) Prafull Joshi (251110056)
Pranjal (251110058) Sarthak Dumbre (251110064)
Vishal Junjare (251110085)

1 Introduction

This assignment focuses on implementing a parallel 3D stencil computation using MPI over a decomposed domain of size $px \times py \times pz$. Each process operates on a sub-domain of size $nx \times ny \times nz$ and performs halo exchange followed by stencil computation for T time steps.

Additionally, each process computes the count of isovalues (zero-crossings relative to a given isovalue) for each field, and results are aggregated at the root process.

2 Code Logic

2.1 Domain Decomposition

The global domain is decomposed into a 3D Cartesian grid of processes. Each process determines its coordinates (x, y, z) using its rank.

Neighbours are identified in six directions:

- East, West
- North, South
- Front, Back

2.2 Data Initialization

Each process initialises its local data using the given random formula:

$$data[i][j] = \frac{rand() \cdot (rank + 1)}{110426.0 + i + j}$$

This ensures different data distributions across processes.

2.3 Halo Exchange

At every time step, each process exchanges boundary data with its neighboring processes so that stencil computation can be performed correctly across sub-domain boundaries. Non-blocking communication is used to initiate all data transfers, allowing communication to proceed while other operations are prepared.

For the X-direction, a derived datatype is used to directly send and receive planes of data. For the Y and Z directions, boundary values are first packed into contiguous buffers before being communicated. Once all communication operations are completed, the received data is unpacked into the halo (ghost) regions of the local grid.

2.4 Stencil Computation

After the halo exchange is completed, each process updates its local grid using a 7-point stencil. The value at each grid point is computed as the average of the point itself and its available neighboring points.

For interior points, all six neighbors are available locally. For boundary points, values from neighboring processes (received through halo exchange) are used when applicable. If a neighbor does not exist, it is simply excluded from the computation, and the averaging is adjusted accordingly.

2.5 Isovalue Counting

Instead of exact equality, the algorithm detects crossings:

$$(a - iso)(b - iso) < 0$$

This checks whether the isovalue lies between two neighbouring grid points.

Only 3 directions are checked (X, Y, Z) to avoid redundant counting.

2.6 Reduction

Each process computes local counts and performs a reduction operation with summation across all processes.

The root process prints results for each timestep.

3 Code Optimizations

- **Non-blocking communication:** Halo exchange is performed using non-blocking communication, which allows overlap between communication and computation. This helps in reducing idle waiting time during data transfer between processes.
- **Derived datatype usage:** A derived datatype is used for transferring data in the X-direction, which avoids manual packing and improves efficiency of communication.
- **Efficient data transfer:** For Y and Z directions, data is packed into contiguous buffers before communication. This ensures better memory access patterns and reduces communication overhead.
- **Memory optimization:** Instead of copying arrays after each time step, pointer swapping is used. This significantly reduces memory operations and improves performance.
- **Reduced redundant computation:** Isovalue counting is restricted to required directions only, which avoids duplicate checks within a process.
- **Boundary handling:** Only valid neighbors are considered during stencil computation, and the denominator is adjusted accordingly. This avoids unnecessary computations at boundaries.

4 Experimental Setup

Configurations used:

- Processes: 32, 48, 64, 96
- Grid sizes: 120^3, 240^3

- $d = 7$, $T = 5$, $F = 2$
- Each experiment repeated 5 times

5 Results

5.1 Performance Results

Experiments were conducted using process counts $P = 32, 48, 64, 96$ and grid sizes $N = 120^3$ and $N = 240^3$. Each configuration was executed five times and the total execution time was recorded.

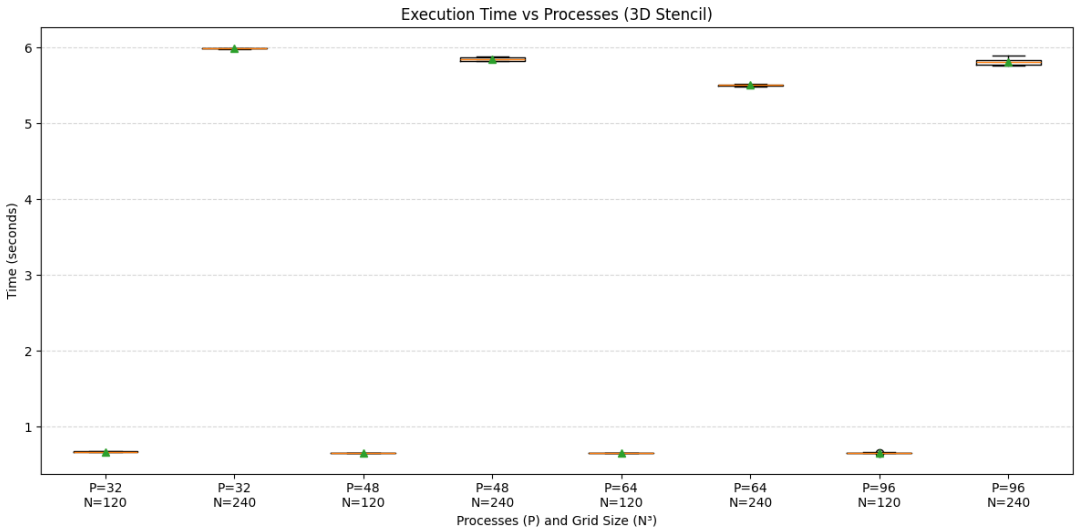

Figure 1: Execution time distribution for different process counts and grid sizes. Each box represents five runs.

Figure 1 shows the execution time distribution across all configurations. The x-axis represents combinations of process count and grid size, while the y-axis shows execution time in seconds. Each boxplot summarizes five runs, with the median shown by the central line and the interquartile range represented by the box.

The average execution times computed from the experimental data are summarized in Table 1.

Processes (P)	$N = 120^3$ (s)	$N = 240^3$ (s)
32	0.668	5.987
48	0.653	5.846
64	0.655	5.502
96	0.659	5.811

Table 1: Average execution time (seconds) over five runs for each configuration.

The results show very low variation across runs for all configurations, indicating stable execution and consistent communication behavior.

5.2 Isovalue Results

The computed isovalue counts for both fields remain consistent across all process counts, confirming correctness of the parallel implementation. Table 2 shows representative values for $P = 32$.

Timestep	Field 1	Field 2
1	311523	312971
2	68905	70422
3	15216	15442
4	3271	3081
5	542	602

Table 2: Isovalue counts for $N = 120^3$ (representative, identical across all P).

For larger grid sizes, the counts increase significantly due to the higher number of grid cells, while maintaining consistency across all process configurations.

6 Observations

- **Effect of increasing processes:** Execution time decreases as the number of processes increases from 32 to 64, showing effective parallelization of the workload.
- **Diminishing returns:** Beyond 64 processes, the improvement is not significant and in some cases execution time slightly increases. This indicates that communication overhead starts to dominate at higher process counts.
- **Impact of problem size:** The larger grid size (240^3) shows better scalability compared to 120^3, as the computation workload is higher relative to communication.
- **Stability of execution time:** The variation across multiple runs is very small for all configurations, indicating stable and consistent performance.
- **Isovalue behavior:** The isovalue counts decrease steadily over time steps, which is expected due to repeated stencil updates smoothing the data.
- **Correctness across configurations:** The isovalue counts remain consistent across different process configurations, confirming correctness of the parallel implementation.

7 Conclusion

The implementation successfully performs parallel stencil computation and isovalue counting on a decomposed 3D domain using MPI. The use of non-blocking communication ensures that data exchange between processes is handled efficiently, while maintaining correctness of the computation across sub-domain boundaries.

The results show that the approach scales well up to a moderate number of processes, beyond which communication overhead starts to affect performance. Overall, the implementation demonstrates effective use of parallel programming concepts such as domain decomposition, halo exchange, and collective operations.